



UNIVERSITY OF THE PUNJAB

B.S. 4 Years Program / Sixth Semester – Fall 2023

Paper: Computational Physics-II

Course Code: PHYS-3504

Time: 3 H

THE ANSWERS MUST BE ATTEMPTED ON THE ANSWER SHEET PROVIDED

Q.1. Write short answers of the following Questions in MATLAB.

- To generate an array R. Find out zero & nonzero elements and index numbers of R. Find maximum value and index number of R.
- Write example to show the function of the following.
(a) min() (b) zeros() (c) size() (d) roots() and (e) diff()
- Write syntax with example for the following in MATLAB:
(a) for() loop (b) input() and (c) gtext()
- Create a vector H with values 100 to -1 having decrement 2. Find sum of H, plot H and print H in reverse order.
- To calculate by a function the area and circumference of a circle by taking the values of radius from the user
- Write one example to show the use of: poly(), polyfit() & rand()

Answer the following questions.

(3×10=30)

Q.2.	How randomly generated points can be used to show Brownian motion? Write MATLAB program to simulate Brownian motion of an object for 31 collisions. Plot estimate graph. Write a program to find out factor of a number. Write MATLAB program to evaluate $\int_1^6 (3\sqrt{x} - 0.5)dx$.
Q.3.	Suppose A be a 3x3 matrix. Write MATLAB program which reads in random numbers as entries of the matrix A and calculate (i) sum and average of the all matrix elements, (ii) transpose of the matrix A (also plot the matrix), (iii) also check whether the Matrix A is an identity matrix? (iv) sort the matrix elements, (v) divide matrix rows by its row average. Write program to study the damped harmonic motion (DHM) of a mass attached with a spring using Euler's method with: ($g=9.8 \text{ m/s}^2$, initial position zero and velocity 15 m/s, time step 0.1 sec. and tmax 15 sec., $k = 1 \text{ N/m}$, $m=1\text{kg}$, damping coefficient = 0.5 N/ms,). Print and plot values for time, position, velocity and acceleration. Also suggest changes to write program for Simple harmonic motion.
Q.4.	Write MATLAB program to plot and print time, position, velocity and acceleration values for a spherical object in air drag. The acceleration and other parameters are given by $a = g - k v^2$ where $k = c \pi \rho r^2 / 2 m$ with the conditions: $g = 9.8 \text{ m/sec}^2$, $c=0.46$ (drag constant), $\rho = 1.2 \text{ kg/m}^3$, $r=1\text{m}$, $v=0 \text{ m/sec}$, $h = 0.1 \text{ sec}$ and $t_{\text{max}}=2.5\text{sec}$. Write code to solve for the system of equations for x, y and z by using two different methods. Such that $2x+y+2z=17$, $4x-3y+8z=6$ and $x-3y+z=6$.

BS4Y

Computational Physics - ii (Fall-2023)

Short Questions :-

(i)

```
» % Create an array R
R = [0 3 0 7 2 0 9 4];
Zero-elements = R(R == 0);
Zero-indices = find(R == 0);
nonzero-elements = R(R ~= 0);
nonzero-indices = find(R ~= 0);
[man-value, man-index] = max(R);
disp('Zero elements and their
      indices:');
disp(Zero-elements);
disp(Zero-indices);
disp('Non zero elements and their
      indices:');
disp(nonzero-elements);
disp(nonzero-indices);
disp('maximum value and its
      index:');
disp(man-value);
disp(man-index);
```


(ii)

→ $\therefore$ Define vector A

$$A = [3 \ 5 \ 2 \ 8 \ 4];$$

$$\text{minValue} = \min(A)$$

$$\text{ZeroMatrix} = \text{zeros}(2, 3)$$

$$B = [1 \ 2 \ 3; 4 \ 5 \ 6];$$

$$\text{SizeB} = \text{size}(B)$$

$$C = [1 \ -3 \ 2];$$

$$\text{PolynomialRoots} = \text{roots}(C)$$

$$D = [10 \ 20 \ 30 \ 40];$$

$$\text{difference D} = \text{diff}(D)$$

(iii)

Syntax:-

for index = start-value : end-value

(a) $\therefore$ Statement

end

(b) $[n, y] = \text{ginput}(n)$

(c) $\text{gtext}('text')$

Example :-

» % Create a blank figure window

figure;

for i = 1:3

[n,y] = ginput(1);

plot(n,y,'xo');

gtext(['Point', num2str(i)]);

end

(iv)

» % Create vector H with values from 100 to -1 decreasing by 2

H = 100:-2:-1;

Sum-H = sum(H);

figure;

plot(H)

title('Plot of H');

xlabel('index');

ylabel('value');

H-reverse = flip(H);

disp(H-reverse)

disp(Sum-H)

(v)

```
1. from the user to input radius  
radius = input('Enter radius:');  
A = pi * radius^2;  
C = 2 * pi * radius;  
disp(A);  
disp(C);
```

(vi)

```
x = rand(1,5);  
y = rand(1,5);  
p = Polyfit(x, y, 2);  
disp(p);  
r = roots(p);  
Polynomial = Poly(r);  
disp(Polynomial);  
x_fit = linspace(min(x), max(x),  
100);  
y_fit = polyval(p, x_fit);  
figure;  
Plot(x, y);  
hold on;  
plot(x_fit, y_fit);
```

```
legend show;  
title('Data and Fitted polynomial'  
);
```

```
xlabel('x');
```

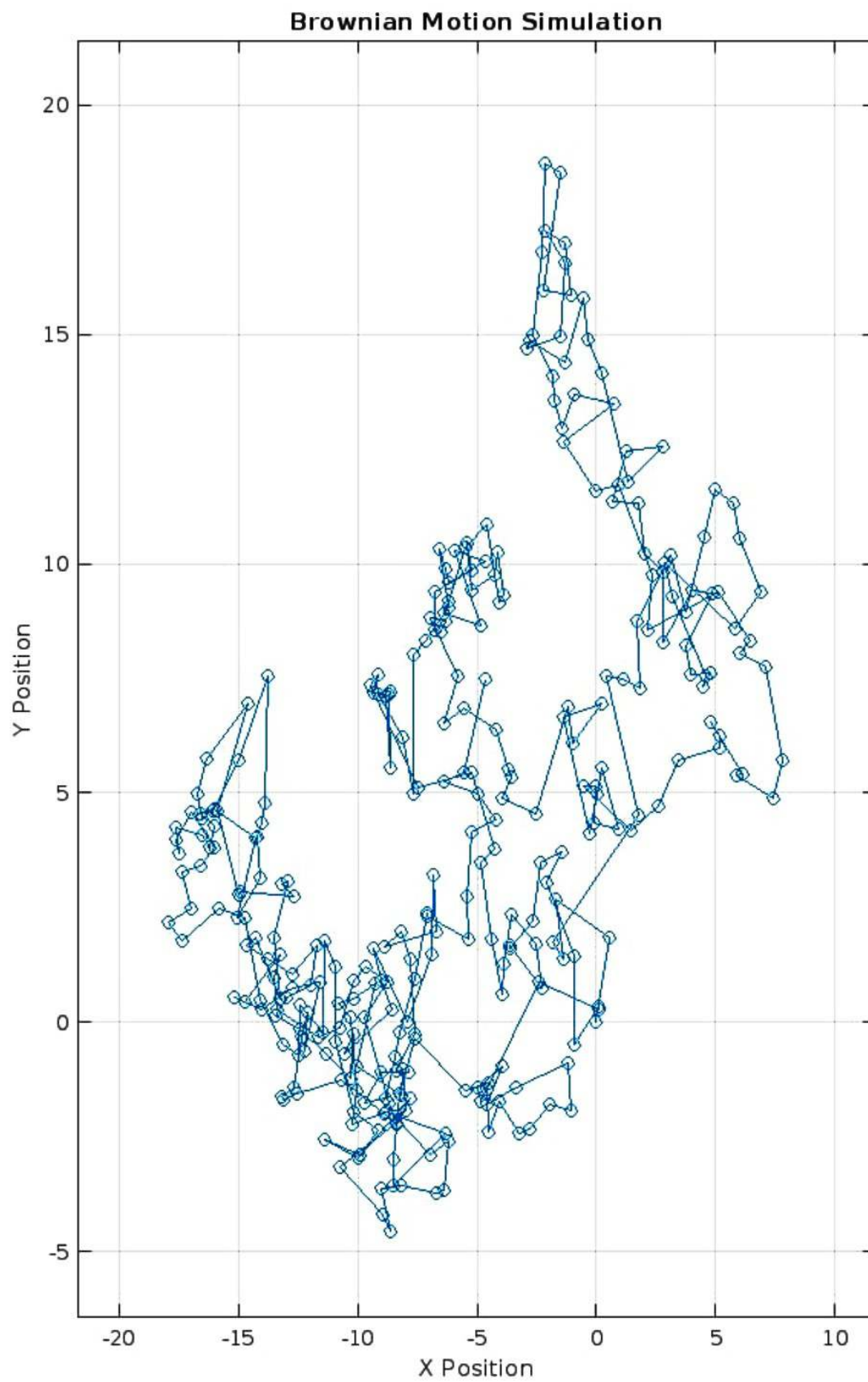
```
ylabel('y');
```

```
grid on;
```


1st long part (a)

```
>> n = 31 ;
    x = zeros(1, n);
    y = zeros(1, n);
    for i = 2:n
        daX = randn;
        daY = randn;
        x(i) = x(i-1) + daX;
        y(i) = y(i-1) + daY;
    end
    figure;
    plot(x, y, '-o');
    title('Brownian Motion
Simulation');
    xlabel('X Position');
    ylabel('Y Position');
    grid on;
    figure;
    d = sqrt(x.^2 + y.^2);
    plot(1:n, d, '-o');
    title('Estimated Displacement
from Origin');
    xlabel('Number of Steps');
    ylabel('Displacement');
    grid on
```

Output



1st long part (b)

```
>> f = @(x) 3*sqrt(x) - 0.5;  
    a = 1;  
    b = 4;  
    D= integral(f, a, b);  
    disp(D)  
  
    12.5000
```

2nd long part (a)

```
>> % 1. Generate a 3x3 matrix A with
    random numbers between 0 and 10
    A = randi([0, 10], 3, 3);
    disp(A);
    S= sum(A(:));
    avg = mean(A(:));
    disp(S)
    disp(avg);
    B= A';
    disp(B);
    figure;
    imagesc(A);
    colorbar;
    title('Matrix A')
    is_identity = isequal(A, eye(3));
    if is_identity
        disp('Matrix A is an identity
matrix. ');
    else
        disp('Matrix A is NOT an
identity matrix. ');
    end
    C=sort(A(:));
    disp(C);
    row_averages = mean(A, 2);
    A_divided = A ./ row_averages;
    disp(A_divided)
```


Output

10	10	1
1	5	4
10	8	10

59

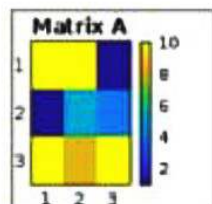
6.5556

10	1	10
10	5	8
1	4	10

Matrix A is NOT an identity matrix.

1
1
4
5
8
10
10
10
10

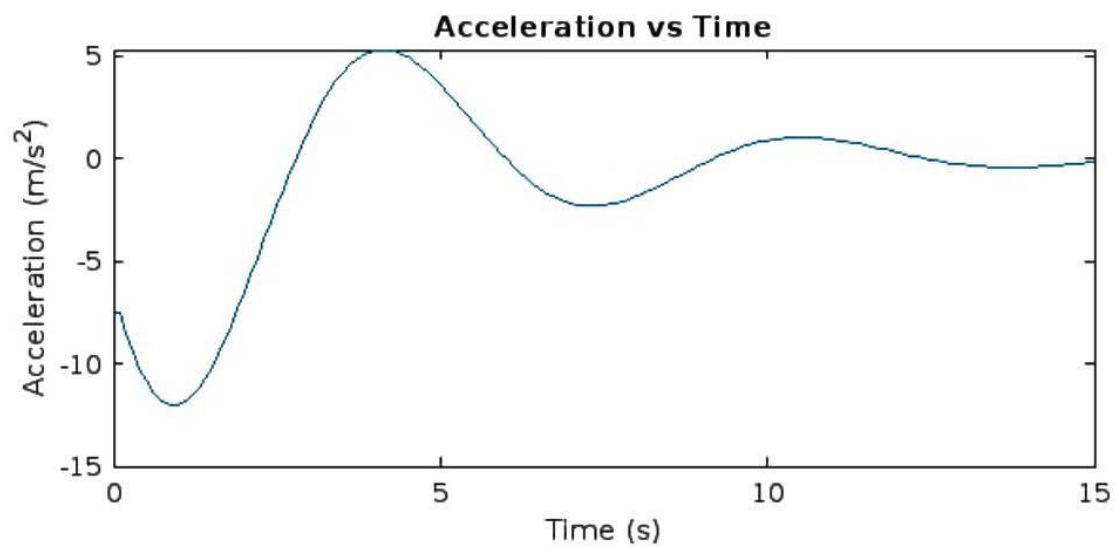
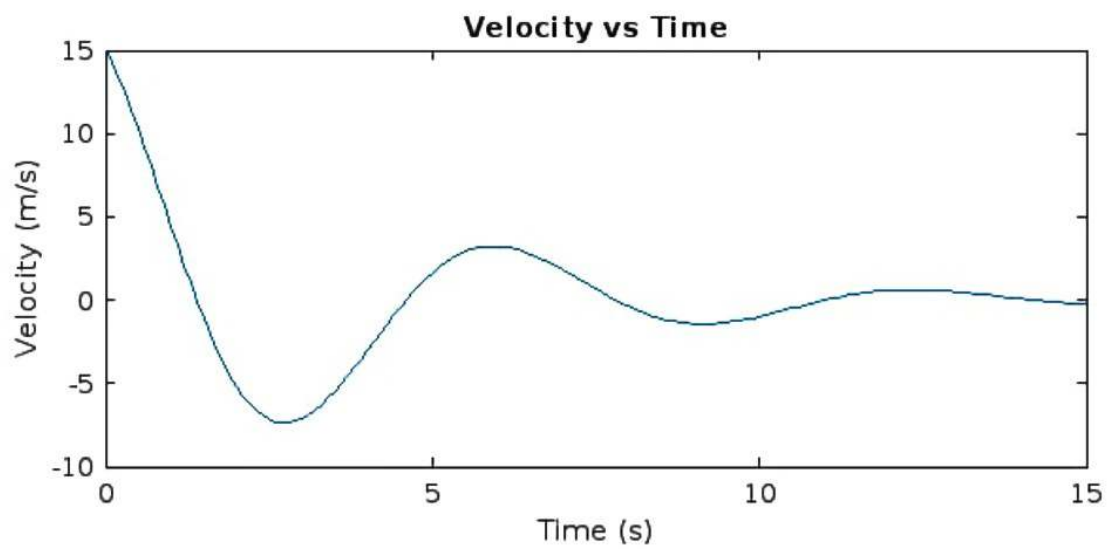
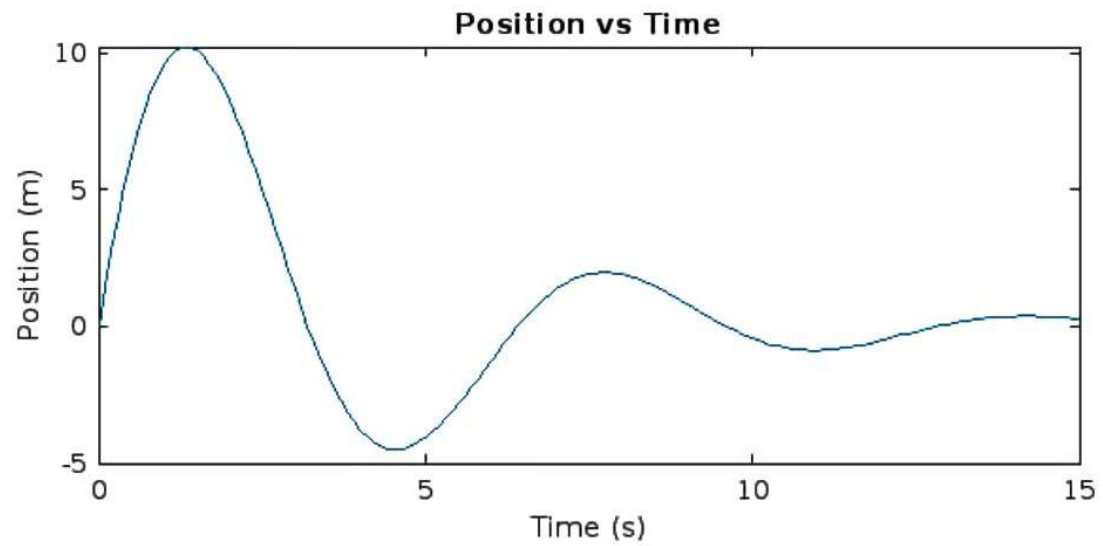
1.4286	1.4286	0.1429
0.3000	1.5000	1.2000
1.0714	0.8571	1.0714



2nd long part (b)

```
>> % DHM
g = 9.8;
k = 1;
m = 1;
c = 0.5;
dt = 0.1;
tmax = 15;
x = 0;
v = 15;
time = 0:dt:tmax;
n = length(time);
x_vals = zeros(1, n);
v_vals = zeros(1, n);
a_vals = zeros(1, n);
x_vals(1) = x;
v_vals(1) = v;
a_vals(1) = -(k/m)*x - (c/m)*v;
for i = 2:n
    a = -(k/m)*x - (c/m)*v;
    v = v + a*dt;
    x = x + v*dt;
    x_vals(i) = x;
    v_vals(i) = v;
    a_vals(i) = a;
end
disp('Time (s)      Position (m)
Velocity (m/s)     Acceleration (m/s^2)')
for i = 1:n
    fprintf('%8.2f      %12.4f
%13.4f      %18.4f\n', time(i), x_vals(i),
v_vals(i), a_vals(i))
end
figure;
subplot(3,1,1);
plot(time, x_vals);
xlabel('Time (s)');
ylabel('Position (m)');
title('Position vs Time');
subplot(3,1,2);
plot(time, v_vals);
xlabel('Time (s)');
ylabel('Velocity (m/s)');
title('Velocity vs Time');
subplot(3,1,3);
plot(time, a_vals);
xlabel('Time (s)');
ylabel('Acceleration (m/s^2)');
title('Acceleration vs Time')
```


Output



3rd long part (a)

```
>> % under gravity and drag force
g = 9.8;
Cd = 0.46;
rho = 1.2;
r = 1;
v0 = 0;
h = 0.1;
tmax = 2.5;
A = pi * r^2;
k = Cd * (rho * A) / (2 * 1);
t = 0:h:tmax;
N = length(t);
v = zeros(1, N);
x = zeros(1, N);
a = zeros(1, N);
v(1) = v0;
x(1) = 0;
for i = 2:N
    a(i-1) = g - k * v(i-1)^2;
    v(i) = v(i-1) + a(i-1) * h;
    x(i) = x(i-1) + v(i-1) * h;
end
a(N) = g - k * v(N)^2;
figure;
subplot(3,1,1);
plot(t, x);
xlabel('Time (s)');
ylabel('Position (m)');
title('Position vs Time');
subplot(3,1,2);
plot(t, v);
xlabel('Time (s)');
ylabel('Velocity (m/s)');
title('Velocity vs Time');
subplot(3,1,3);
plot(t, a);
xlabel('Time (s)');
ylabel('Acceleration (m/s^2)');
title('Acceleration vs Time');
for i = 1:N
    fprintf( t(i), x(i), v(i), a(i));
end
```


3rd long part (b)

```
>> % Cramer rule
A = [2 1 2;
      4 -3 8;
      1 -3 1];
det_A = det(A);
Ax = A;
Ax(:,1) = b;
det_Ax = det(Ax);
Ay = A;
Ay(:,2) = b;
det_Ay = det(Ay);
Az = A;
Az(:,3) = b;
det_Az = det(Az);
x= det_Ax / det_A;
y= det_Ay / det_A;
z= det_Az / det_A;
disp('x,y,z')
```

4 -3 8;

3rd long part (b) second method

```
>> A = [2 1 2;  
        4 -3 8;  
        1 -3 1];  
b = [17; -6; -6];  
  
x = inv(A) * b;  
  
disp(x);
```